Haoquan Wang
u1570975
2026-3-19

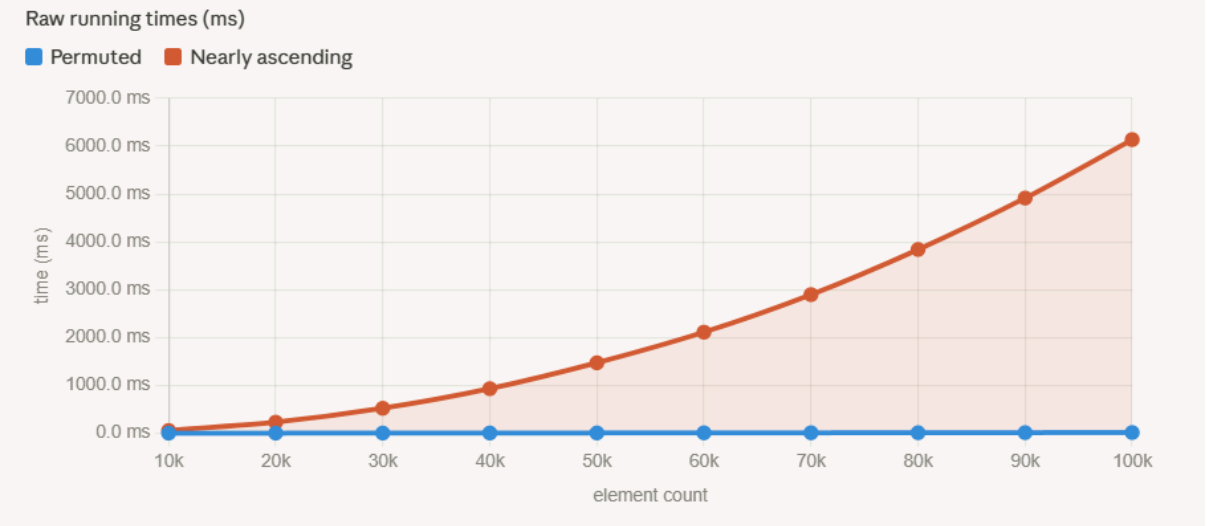
Assignment 8: BinarySearchTree

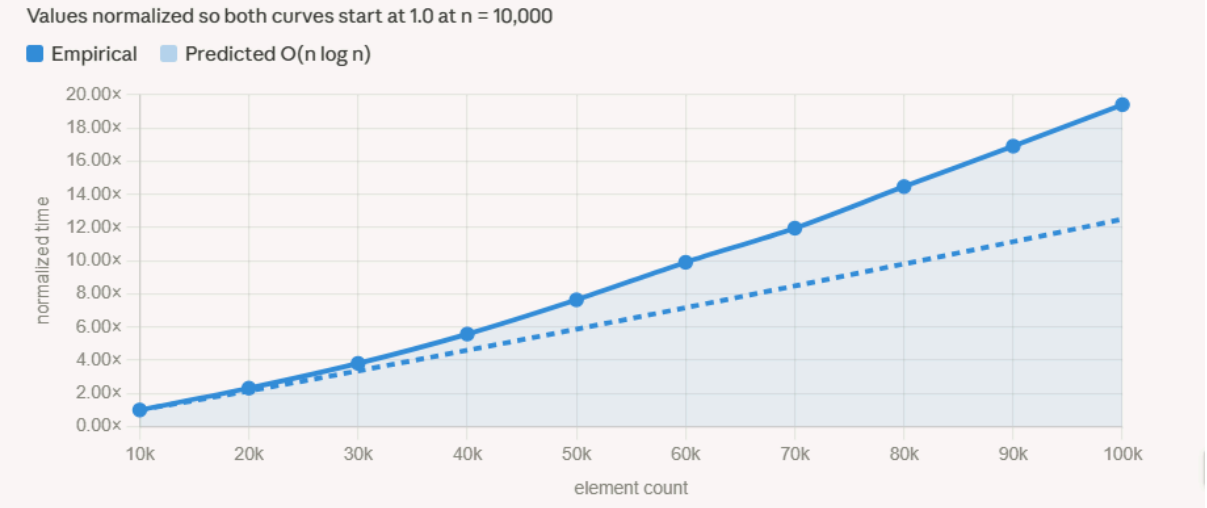
1. This assignment was completed individually without a partner.

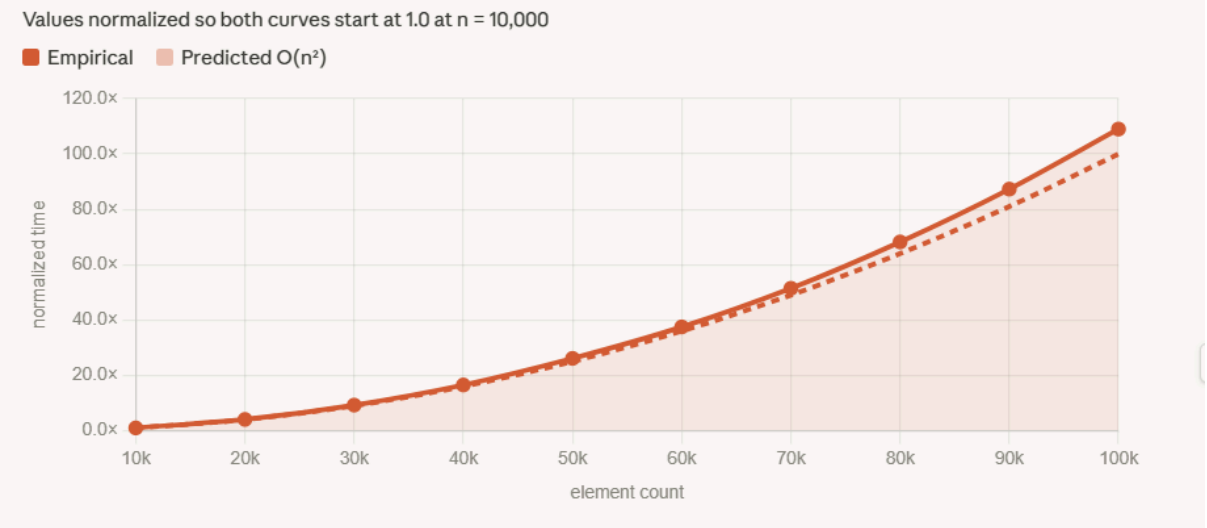
2. For permuted input, the predicted complexity of addAll is O(n log n). With random insertion order, the BST remains approximately balanced, so each add traverses O(log n) nodes on average. The empirical data confirms this: from n = 10,000 to n = 100,000, time grows from 0.70 ms to 13.61 ms, a ratio of 19.4. The predicted O(n log n) ratio is (100,000 × $\log_2$100,000) / (10,000 × $\log_2$10,000) ≈ 12.5. The empirical ratio is somewhat higher due to cache effects at larger sizes, but the growth is clearly sub-quadratic and consistent with O(n log n).

   For nearly ascending input, the predicted complexity is O(n^2). Each new element is almost always larger than all previous ones, so it is placed at the rightmost leaf, and the tree degenerates into a right-skewed linked list. Time grows from 56.32 ms to 6,139.14 ms (ratio ≈ 109), close to the predicted O(n^2) ratio of 100. This matches expectations.

   The contrast is striking: at n = 100,000, nearly ascending insertion is approximately 450× slower than permuted insertion, caused entirely by the degenerate tree structure.



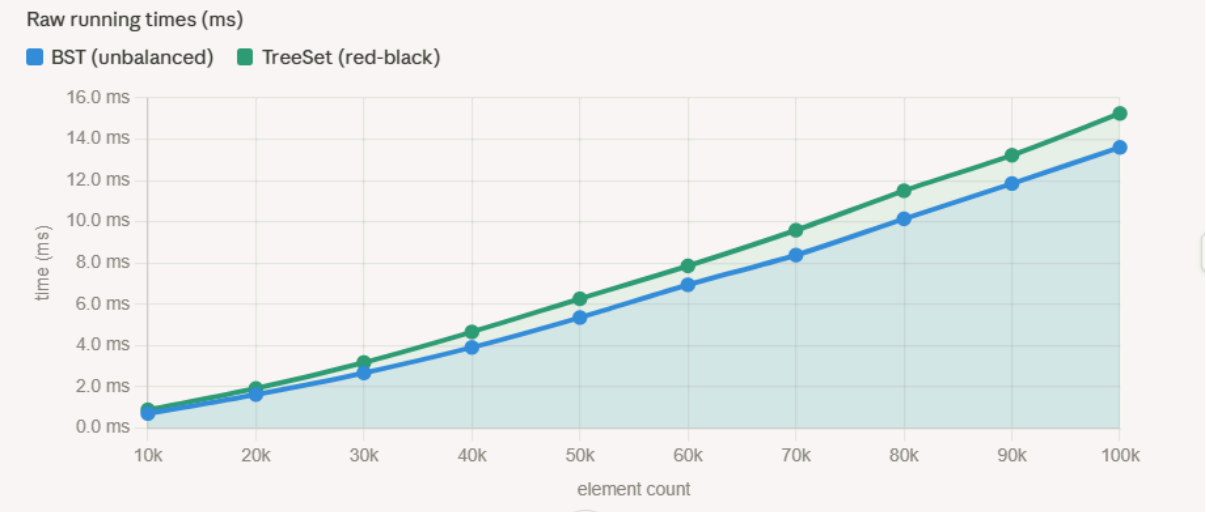


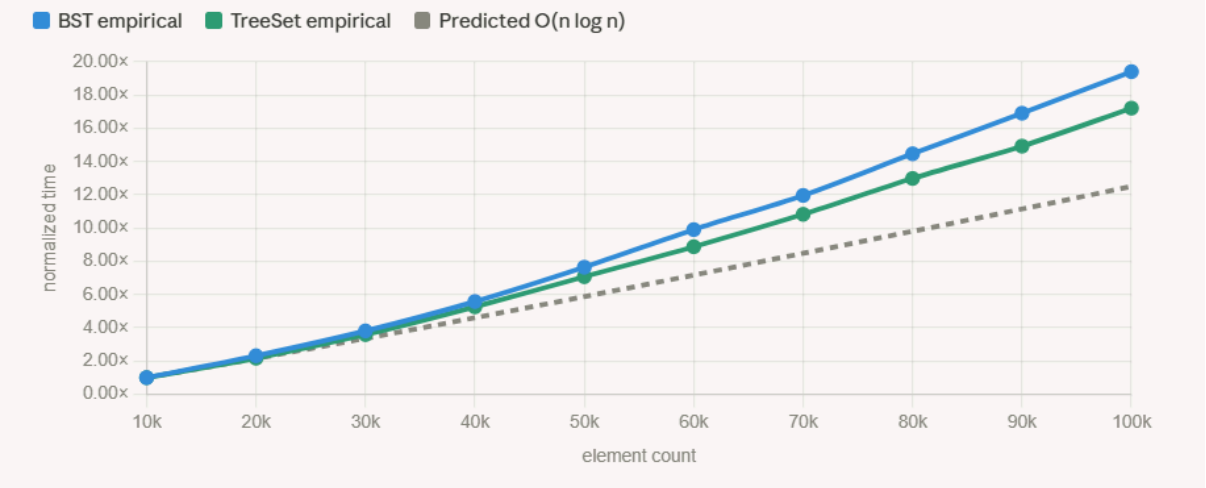


3. A BST is a reasonable dictionary structure because contains runs in O(log n) average time for a balanced tree. However, inserting words in alphabetical order causes the BST to degenerate into a right-skewed linked list of height n, reducing every lookup to O(n) — no better than a plain list.

Two fixes: (1) shuffle the words into random order before insertion to restore expected O(log n) height; or (2) use a self-balancing BST such as a red-black tree (Java's TreeSet), which maintains O(log n) height regardless of insertion order.

4. Both BST and TreeSet show O(n log n) growth for permuted input, with very similar absolute times — TreeSet is roughly 10–15% slower. This matches expectations: for random input the unbalanced BST stays approximately balanced, so both structures cost O(log n) per insertion. The check analysis ratio for TreeSet is 15,251.6 / 886.2 ≈ 17.2, consistent with the predicted ratio of ≈ 12.5. TreeSet's small additional overhead comes from the rotations and color-flipping needed to maintain the red-black tree invariant, which are infrequent and cheap for random input.

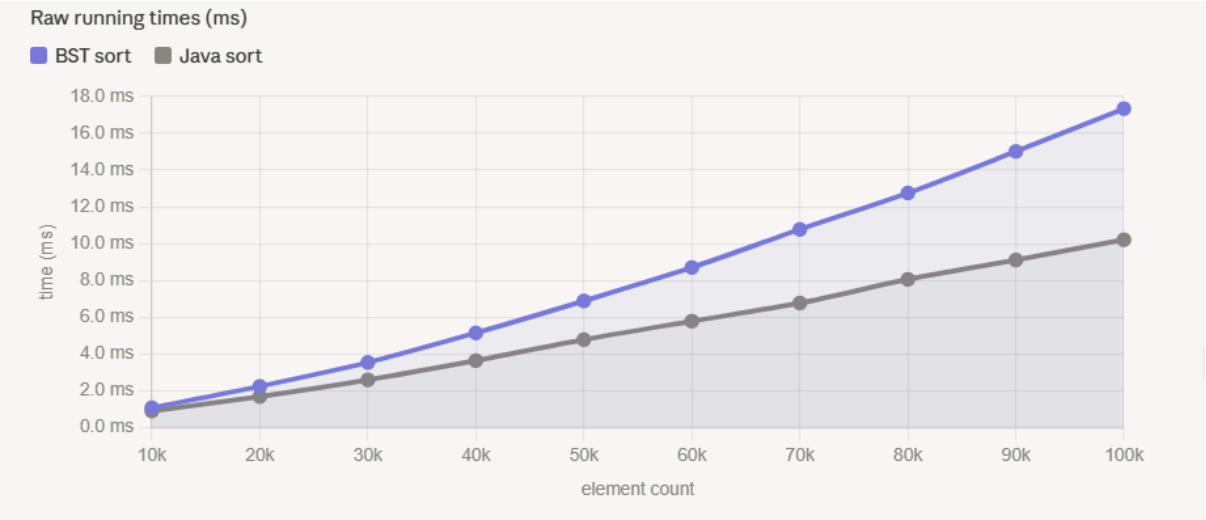


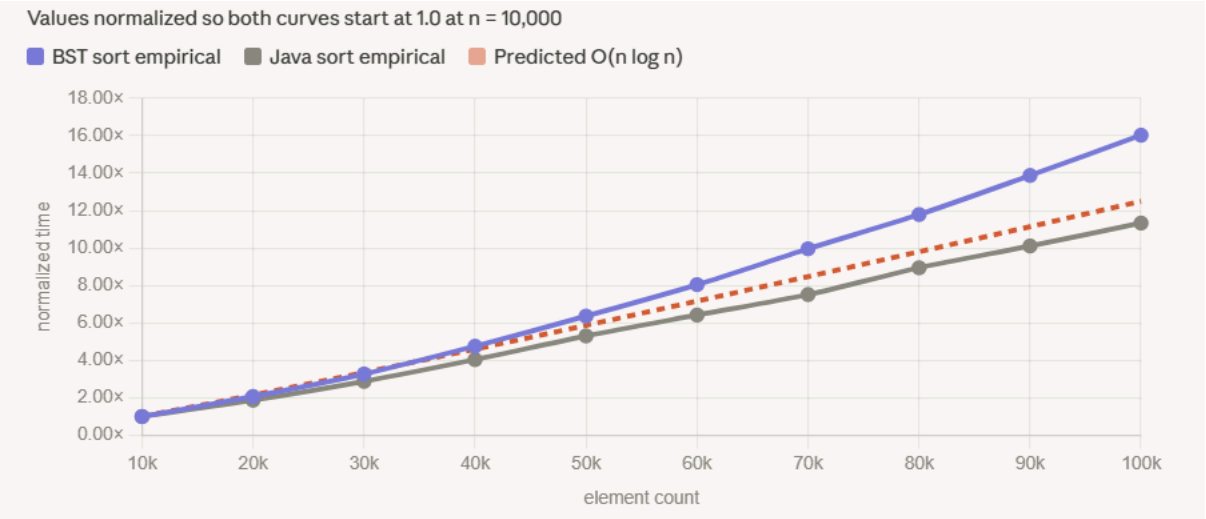


5. For nearly ascending input, the unbalanced BST degrades to O(n²) as shown in Question 2, while TreeSet would remain O(n log n) due to red-black tree rebalancing. At n = 100,000, the unbalanced BST already takes ~6,139 ms for nearly ascending input, while TreeSet with random input took only ~15 ms.

TreeSet with nearly ascending input would remain in a similar range, making it thousands of times faster than the unbalanced BST at large n.

6. Both methods exhibit O(n log n) growth for permuted input. SortViaBST.sort has two O(n log n) phases: inserting all elements via addAll, then extracting them in order via repeated first() and remove(). Java's Collections.sort uses Timsort, also O(n log n). Check analysis ratios: BST sort 17,347.7 / 1,081.8 ≈ 16.0; Java sort 10,215.6 / 901.3 ≈ 11.3 — both consistent with the predicted ratio of ≈ 12.5. Java sort is consistently 40–70% faster due to better cache locality (contiguous array vs. pointer-linked nodes) and the fact that Timsort is a single-pass algorithm while BST sort makes two full passes.





7. For nearly ascending input, BST sort would degrade to O(n²) since the addAll phase produces a degenerate tree. Java's Timsort, by contrast, detects existing ascending runs and approaches O(n) time for nearly sorted input. The performance gap would be far larger than observed for permuted input —

BST sort would be orders of magnitude slower while Java sort would be near its best case.